

Big Data and Data Science Technologies

Lecture 1 — Introduction



University of
Southern Denmark

Jakob Hviid, PhD
Associate Professor, SDU
September 07, 2026

Agenda

- Who am I?
- Who are you?
- What is this course about?
- Introducing perspectives
 - Non-Functional Requirements
 - Horizontal Scaling vs Vertical Scaling
 - The practical history of storage solutions
 - Modern large-scale deployments in the future
- The Project
- Discord



Who am I?

- **About me:** Jakob Hviid, from 1982. Gamer (650+ games on Steam alone), D&D, professional nerd, loves gadgets.
- **Currently:** Associate Professor (SDU) and Senior Solution Architect (Mjølner Informatics)
- **Education:** PhD in Software Engineering, M.Sc. in Software Engineering, B.Sc. Web Development, A.P. Degree in Computer Science
- **Industry experience:**
 - **Previous career:** typically server and network architecture related work
 - **Current career:** researcher, teacher, data expert
 - **Workplaces include:** SDU, Mjølner Informatics, Danish Energy Agency, Energinet, Hviid Development, Esoft, Blue Ocean Robotics, Scannet, Unik System Design, Carlsberg / Tuborg / Coca-Cola and more
- **Teaching:**
 - Data Management (2 semester bachelor)
 - Big Data and Data Science Technologies (1 semester master)



Course Introduction



B.Sc. != M.Sc.

At a **B.Sc.** the lecturer explains

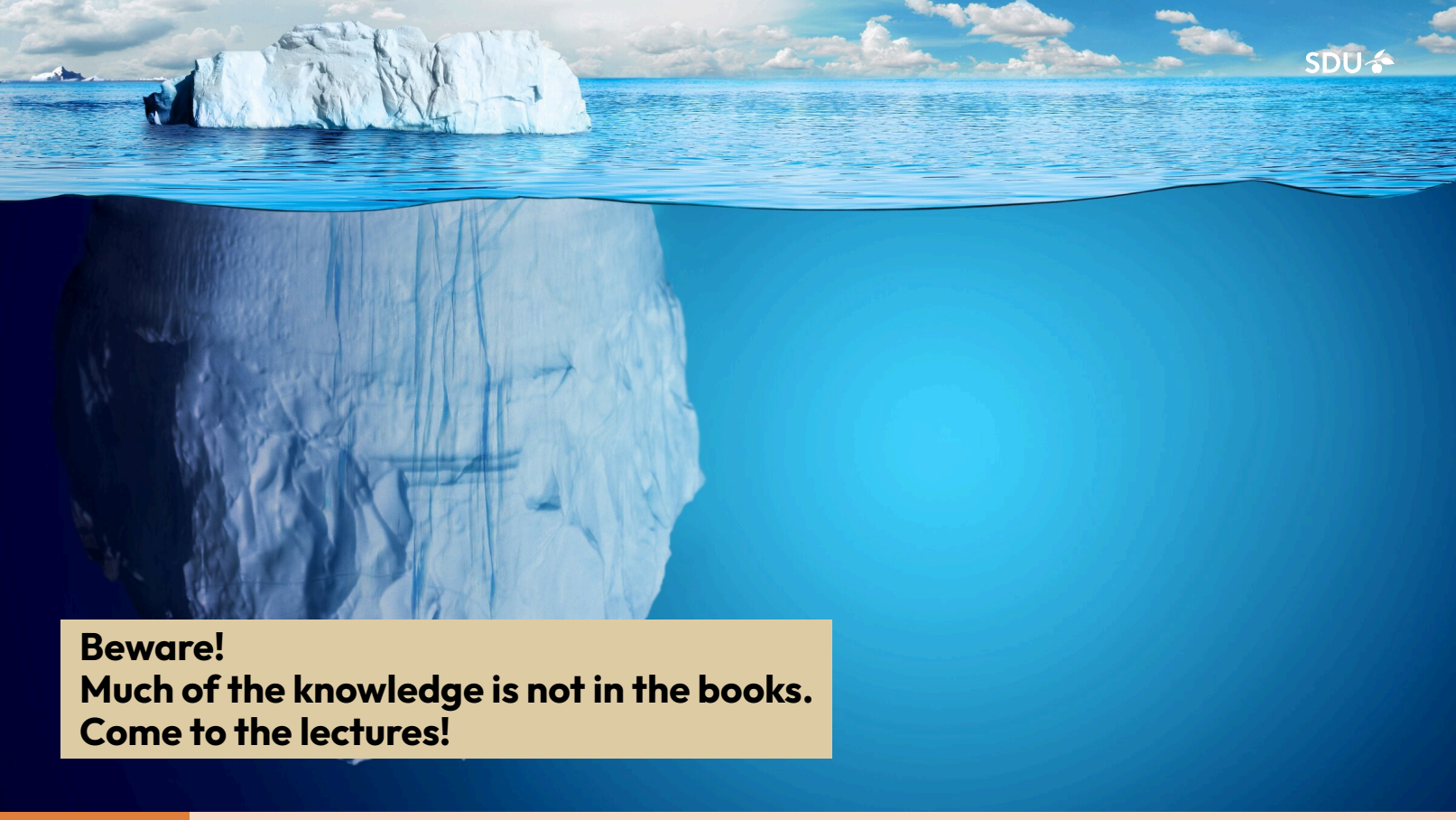
- what the technologies are
- how they are used
- why we are using them

At a **M.Sc.** the lecturer explains

- what technologies exist
- why they are needed
- how they fit together

At a **M.Sc.** the lecturer does **not** explain

- how they are used
- how to write it
- where to find all the information needed

A large iceberg floats in a blue ocean under a cloudy sky. The visible tip of the iceberg is small and jagged, while the submerged part is much larger and smooth. The water surface is slightly wavy.

Beware!
Much of the knowledge is not in the books.
Come to the lectures!



Skills

Analyze and implement a big data pipeline, including:

- dataset selection
- data ingestion
- distributed data storage
- distributed data streaming
- distributed data processing

Competences

After successful completion of the course, the student is able to:

- design large scale storage for industrial, data-intensive applications
- design solutions for processing large data streams
- implement distributed data processing programs
- implement complex analytical queries in big data query languages
- configure, execute and debug programs over the Big Data frameworks



Course Structure (itslearning)

How would you solve this?



You have to receive **1 PB** of data



Process it **on the fly** while you receive it

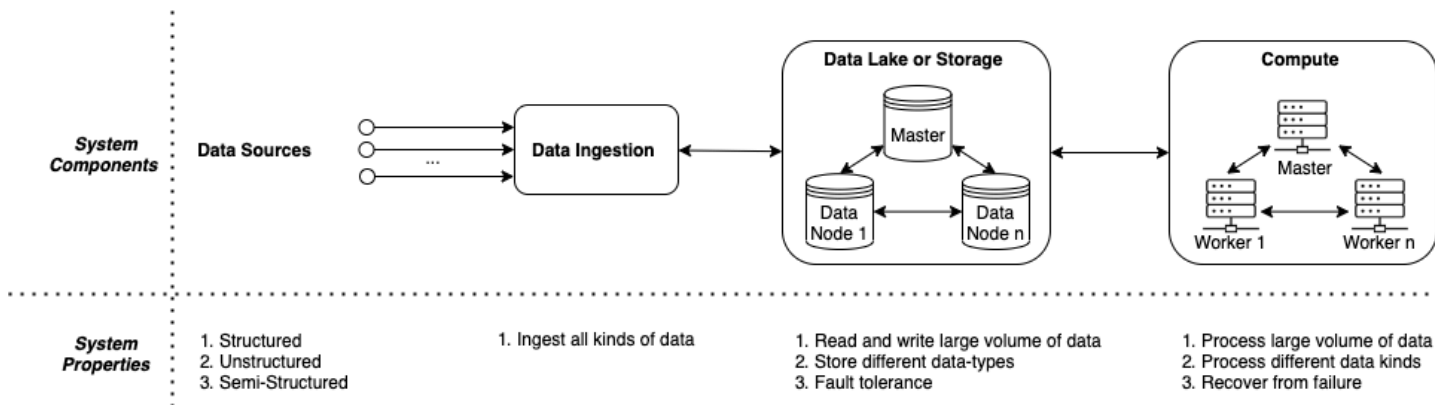


You have to **store all** of the data

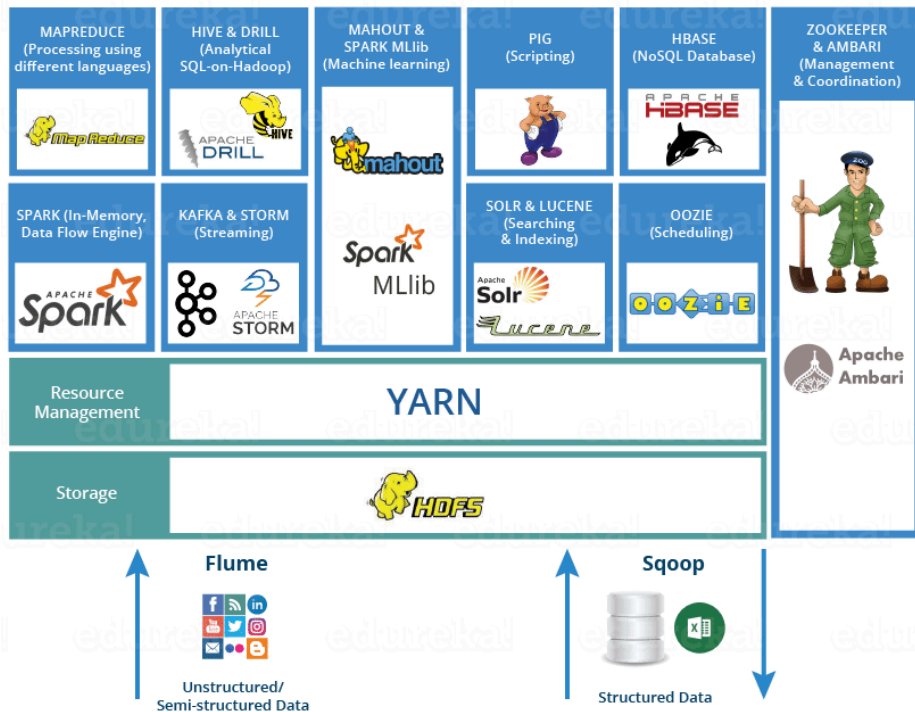


You have to run analytics on this data with a response time of **maximum 3 minutes**

One way to solve this problem



The Apache Stack



Non-Functional Requirements

Just what is a non-functional requirement?



Non-Functional Requirements

- **Availability**
- **Deployability**
- Energy Efficiency
- Integrability
- Modifiability
- **Performance**
- Safety
- Security
- Traceability
- Testability
- Usability
- Documentation



Availability Tactics

Detect Faults

Monitor
Ping/Echo
Heartbeat
Timestamp
Condition Monitoring
Sanity Checking
Voting
Exception Detection
Self-Test

Recover — Preparation and Repair

Redundant Spare
Rollback
Exception Handling
Software Upgrade
Retry
Ignore Faulty Behavior
Graceful Degradation
Reconfiguration

Recover — Reintroduction

Shadow
State Resynchronization
Escalating Restart
Nonstop Forwarding

Prevent Faults

Removal from Service
Transactions
Predictive Model
Exception Prevention
Increase Competence Set

Deployability Tactics

Manage Deployment Pipeline

Scale Rollouts

Script Deployment Commands

Rollback

Manage Deployed System

Manage Service Interactions

Package Dependencies

Toggle Features

Bass, Clements & Kazman, Software Architecture in Practice

Performance Tactics

Control Resource Demand

- Manage Work Requests
- Limit Event Response
- Prioritize Events
- Reduce Computational Overhead
- Bound Execution Times
- Increase Efficiency

Manage Resources

- Increase Resources
- Introduce Concurrency
- Maintain Multiple Copies of Computations
- Maintain Multiple Copies of Data
- Bound Queue Sizes
- Schedule Resources

Bass, Clements & Kazman, Software Architecture in Practice

Production Metrics and Concepts

SLA, SLO, SLI

SLA — Service Level Agreement

The agreement you make with your clients or users.

SLO — Service Level Objectives

The objectives your team must hit to meet that agreement.

SLI — Service Level Indicators

The real numbers on your performance.

What is this?

$$\frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

What is this?

$$\frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

Availability — the share of total time the system is actually working

MTBF is the mean uptime between failures; **MTTR** the mean time to restore service. So the denominator is one whole up-and-down cycle, and the ratio is the part of that cycle spent up.

An MTBF of 99 hours with an MTTR of 1 hour gives $99/100 = \mathbf{99\%}$ availability — the first row of the Error Budget table, two slides on. Cutting MTTR matters as much as raising MTBF.

Incident Metrics

MTBF Mean Time Between Failures

MTTD Mean Time to Detect

MTTR Mean Time to Recovery / Repair / Response / Resolve

MTTA Mean Time to Acknowledge

Error Budget

Availability	Downtime / 90 days	Downtime / year
99.0%	21 hr, 36 min	3 days, 15.6 hr
99.9%	2 hr, 10 min	8 hr, 0 min, 46 sec
99.99%	12 min, 58 sec	52 min, 34 sec
99.999%	1 min, 18 sec	5 min, 15 sec
99.9999%	8 sec	32 sec

Table 4.1, "System Availability Requirements", Bass, Clements & Kazman, Software Architecture in Practice

Severity Levels

Severity	Description	Examples
1	A critical incident with very high impact	A customer-facing service is down for all customers Confidentiality or privacy is breached Customer data loss
2	A major incident with significant impact	A customer-facing service is unavailable for a subset of customers Core functionality (e.g. git push, issue create) is significantly impacted
3	A minor incident with low impact	A minor inconvenience to customers, workaround available Usable performance degradation

Big Data — What and Why?

Big Data is defined in three Vs (yes, there are more)

Volume Large amounts of data.

Variety The data comes in different forms — or from different sources — including traditional databases, images, documents and complex records.

Velocity The content of the data is constantly changing, through the absorption of complementary data collections, through the introduction of previously archived data or legacy collections, and from streamed data arriving from multiple sources.

Big Data Facts

How much data

“Every 2 days we create as much data — **5 exabytes** — as we did from the beginning of time until 2003.”
— Eric Schmidt, Google, **2010**

Today: IDC puts 2025 at **181 zettabytes** created, captured, copied and consumed worldwide. At that rate, those 5 exabytes take about **15 minutes** — not two days.

What bad data costs

“Poor data across businesses and the government costs the U.S. economy **\$3.1 trillion** a year.” — IBM, via Harvard Business Review, **2016**

Today: nobody has re-run the national estimate. The figure usually quoted now is per company: about **\$12.9 million** a year on average. — Gartner, **2021**

IDC counts data created, captured, copied **and** consumed, which is not Schmidt’s measure — the comparison is indicative, not like-for-like

Big Data at Facebook

Based on company statistics [W1] (2012), **per day**:

- 500+ TB of new data into the databases
- 2.5 billion content items shared
(status updates, wall posts, photos, videos, comments)
- 2.7 billion likes
- 300 million photos uploaded

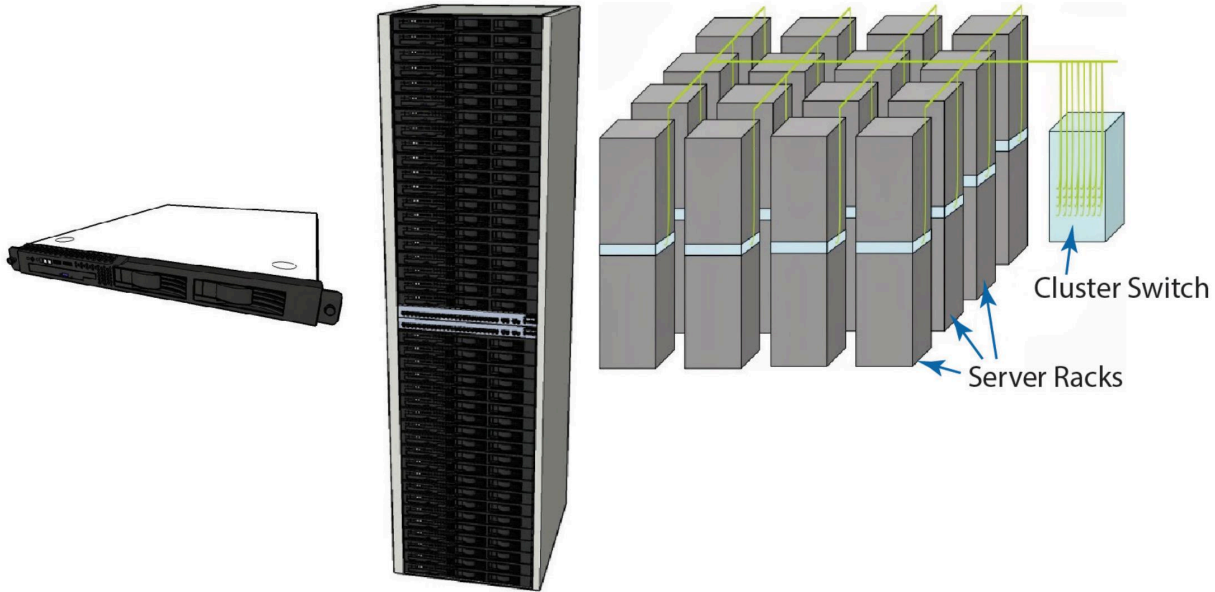


Big Data Storage and Processing

To store and process this big data, we need a new software stack:

- distributed, over networked computing clusters
- a new **Distributed File System** (DFS)
- ...which is extremely fault-tolerant
- new **programming frameworks** over a DFS

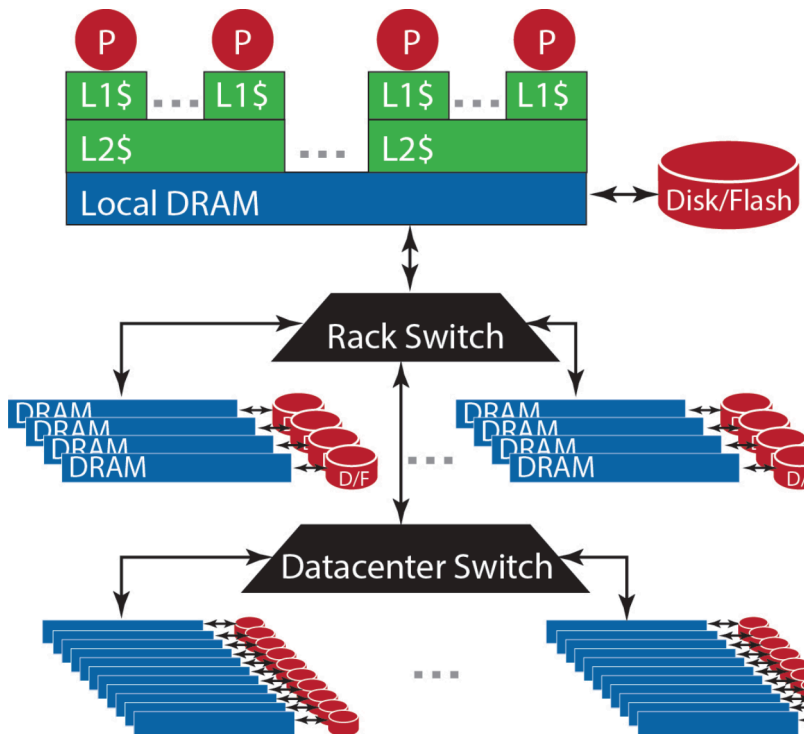
The cluster is the new computer



(left) commodity server — (middle) rack with Ethernet switch and 40–80 servers — (right) cluster with a cluster-level Ethernet switch or router



Google data center, Council Bluffs, Iowa [W2]



Cluster memory: size, latency and bandwidth

One server

DRAM: 16 GB, 100 ns, 20 GB/s

Disk: 2 TB, 10 ms, 200 MB/s

Flash: 128 GB, 100 μ s, 1 GB/s

Local rack (80 servers)

DRAM: 1 TB, 300 μ s, 100 MB/s

Disk: 160 TB, 11 ms, 100 MB/s

Flash: 20 TB, 400 μ s, 100 MB/s

Cluster (30 racks)

DRAM: 30 TB, 500 μ s, 10 MB/s

Disk: 4.80 PB, 12 ms, 10 MB/s

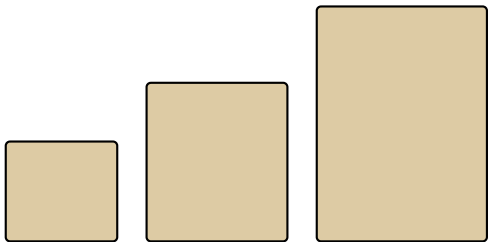
Flash: 600 TB, 600 μ s, 10 MB/s

Google numbers [W3] — an example; the
numbers are outdated

Horizontal vs Vertical Scaling

Horizontal vs Vertical Scaling

Vertical — increase the size of the instance



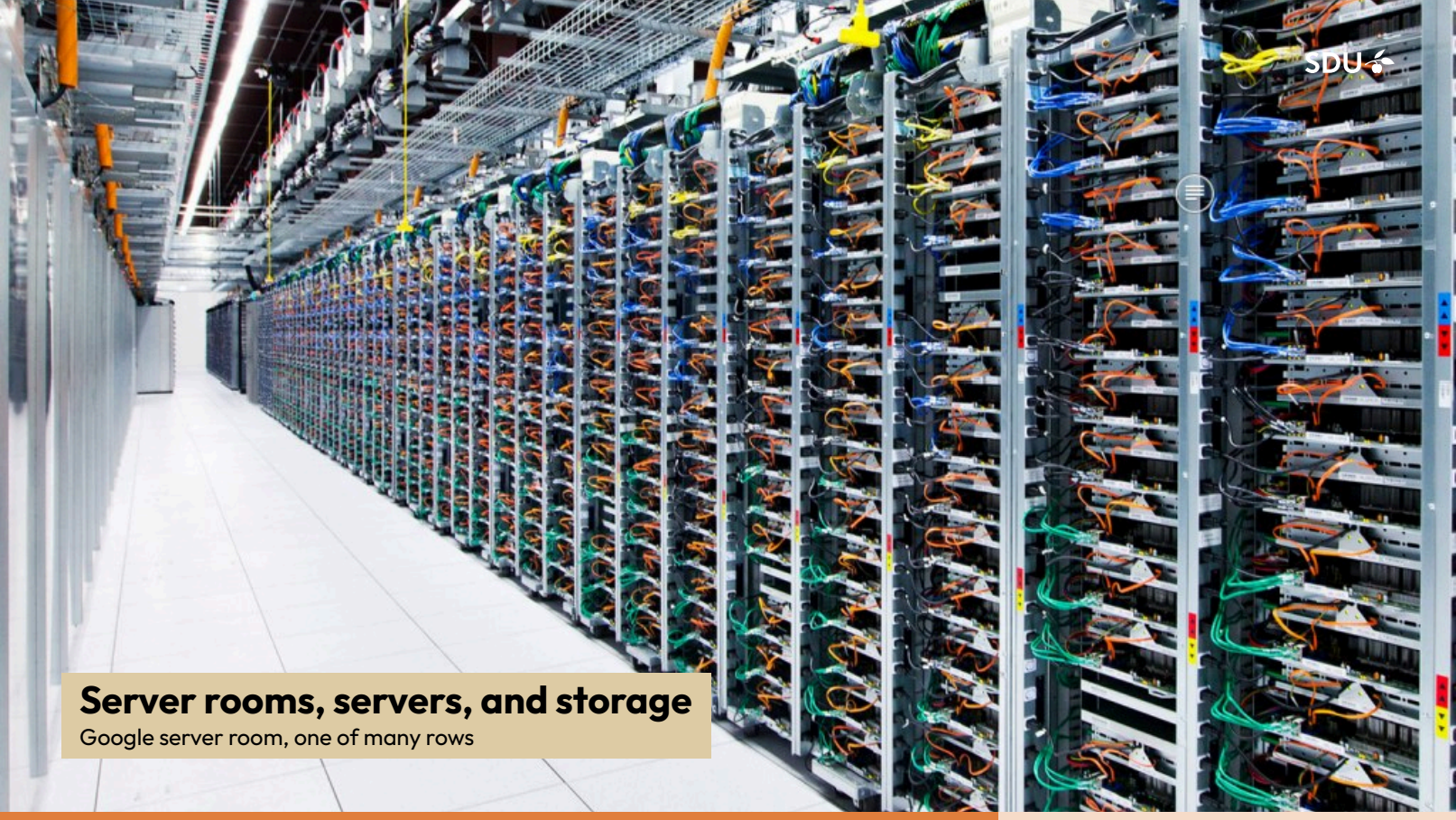
- increasingly expensive to get larger hardware
- requires no code changes

Horizontal — add more instances



- less expensive hardware
- allows for large-scale systems
- need to change software for a distributed architecture, which leads to more complex code
- need for load balancing

Do you know examples of these?



Server rooms, servers, and storage

Google server room, one of many rows

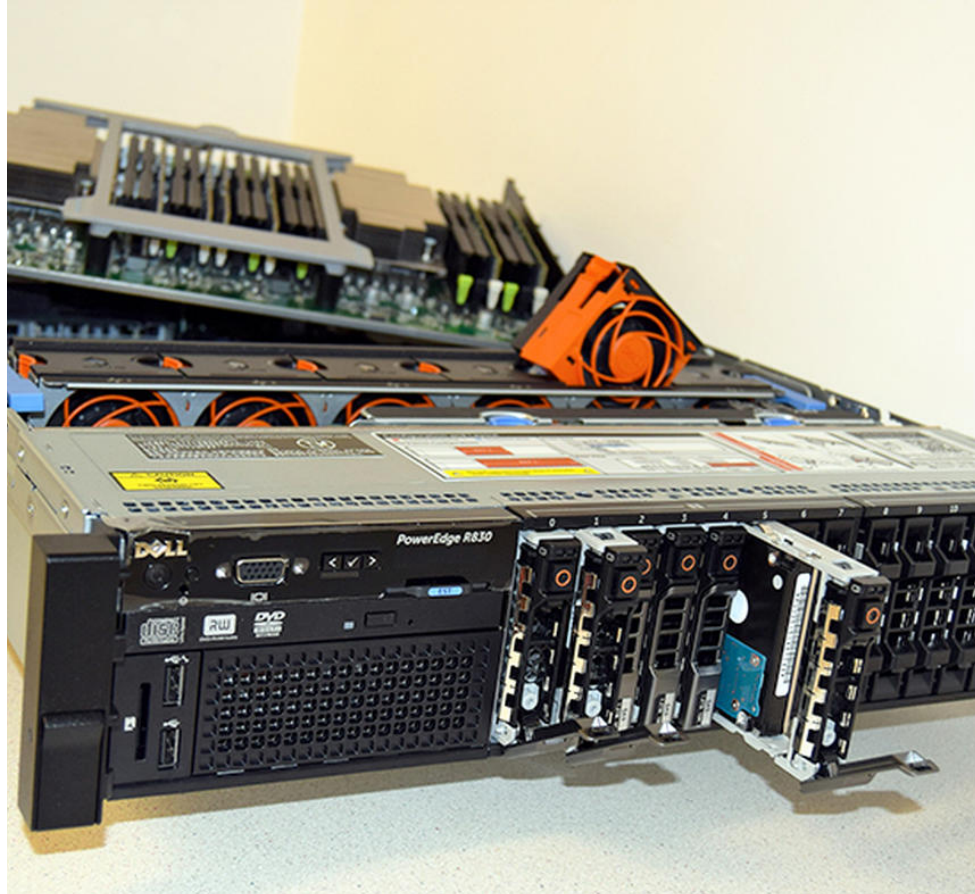
The Rack

- A standard rack is **42U** (rack units)
- 1U equates to 44.45 mm — one server on the picture
- Servers may take up several U
- This equates to roughly **1.9 m** in height



Dell 2U Server

- “Commodity hardware”
- SAS, 15 000 RPM, etc.



RAID Levels

 **BREAKDOWN OF COMMON RAID LEVELS**

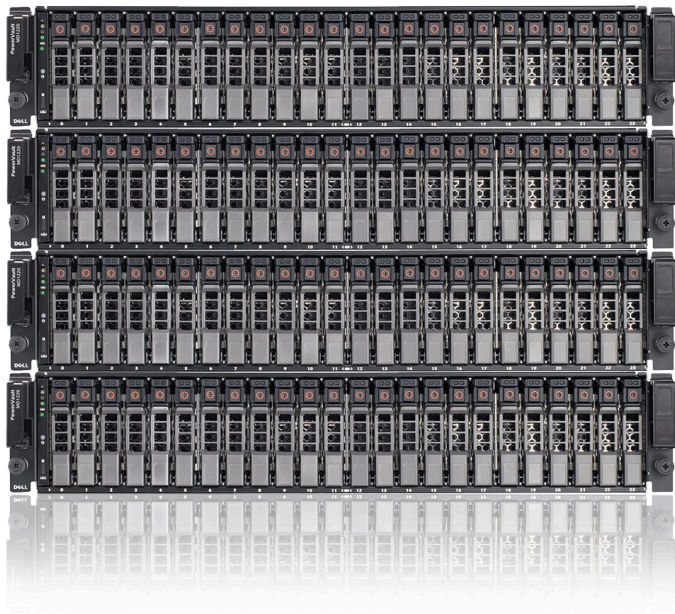
RAID LEVEL	METHOD	HARDWARE / SOFTWARE	MINIMUM # OF DISKS	COMMON USAGE	PROS	CONS
JBOD	SPANNING		2	INCREASE CAPACITY	COST-EFFECTIVE STORAGE	NO PERFORMANCE OR SECURITY BENEFITS
0	STRIPING		2	HEAVY READ OPERATIONS	HIGH PERFORMANCE (SPEED)	DATA IS LOST IF ONE DISK FAILS
1	MIRRORING		2	STANDARD APP SERVERS	FAULT TOLERANCE, HIGH READ PERFORMANCE	LAG FOR WRITE OPS, REDUCED STORAGE (BY 1/2)
5	STRIPING & PARITY		3	NORMAL FILE STORAGE & APP SERVERS	SPEED + FAULT TOLERANCE	LAG FOR WRITE OPS, REDUCED STORAGE (BY 1/3)
6	STRIPING & DOUBLE PARITY		4	LARGE FILE STORAGE & APP SERVERS	EXTRA LEVEL OF REDUNDANCY, HIGH READ PERFORMANCE	LOW WRITE PERFORMANCE, REDUCED STORAGE (BY 2/5)
10 (1+0)	STRIPING & MIRRORING		4	HIGHLY UTILIZED DATABASE SERVERS	WRITE PERFORMANCE + STRONG FAULT TOLERANCE	REDUCED STORAGE (1/2), LIMITED SCALABILITY

What Happened to 2-4 and 6-9?

The RAID levels described above are the most common levels used in enterprise scenarios. The levels in between are highly specialized and only make sense in very specific scenarios.

Expanding with DAS

- Adds storage to a single node server
- Expensive
- Does not add extra processing power
- Controllers become bottlenecks

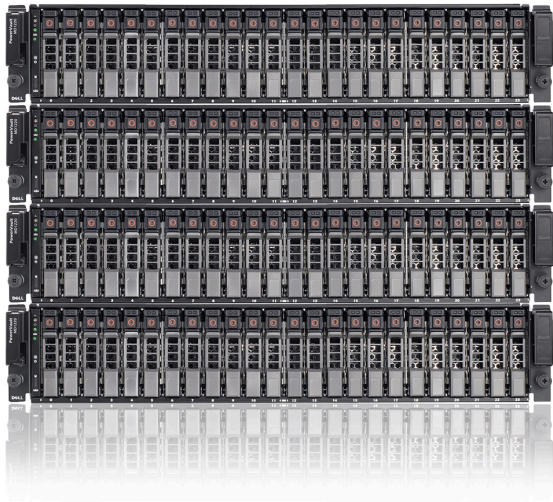


Considerations with traditional RAID

- Disks can be protected at several levels
- What if a **node** dies, not just a disk?
- Replacing disks needs to be handled
- Disks from the same production run often fail together — this can be randomized

Software RAID

- A relatively new phenomenon
- Some products: **ZFS**, **Microsoft Storage Spaces**
- Allows for versatility
 - does not require the same disk sizes
 - does not require a specific RAID controller, so it is easy to move between setups
 - can often leverage RAM and SSD as read/write cache
- Disadvantages
 - RAID 5 calculated in the CPU, not in specialized hardware
 - RAID 5 often not available



SAN

- Virtualized storage
- Managed
- Critical operations
- Extremely expensive (management, hardware, maintenance)
- High performance (RAM and SSD read cache)
- Limited by network speed
- Allows bootable virtualized storage over a network — the OS sees a directly attached disk, so the process is transparent
- Easy to add disk space
- Easy to resize a server's SAN-associated disk without reinstalling
- In some cases deduplication, especially with ZFS
- Storage easy to snapshot and back up

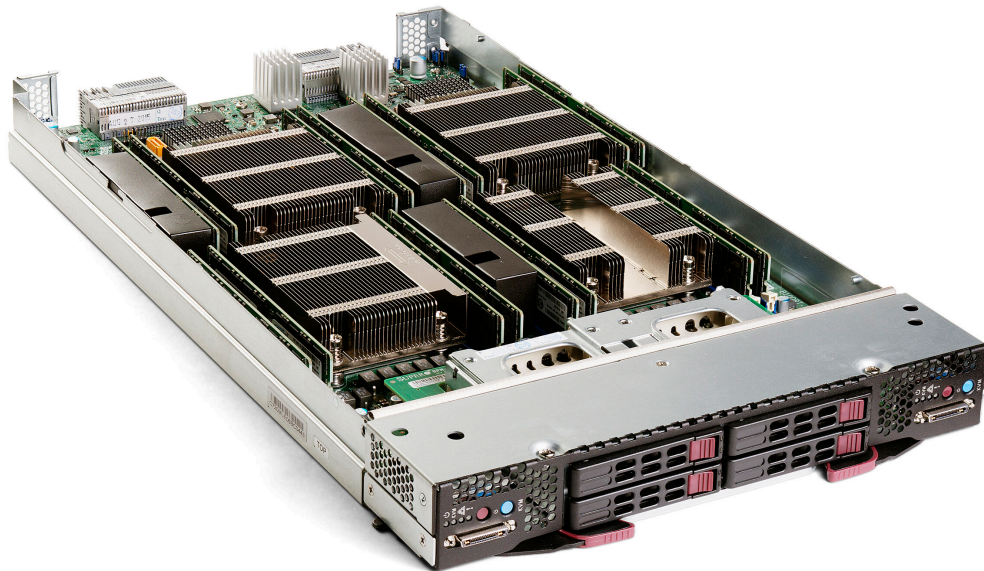




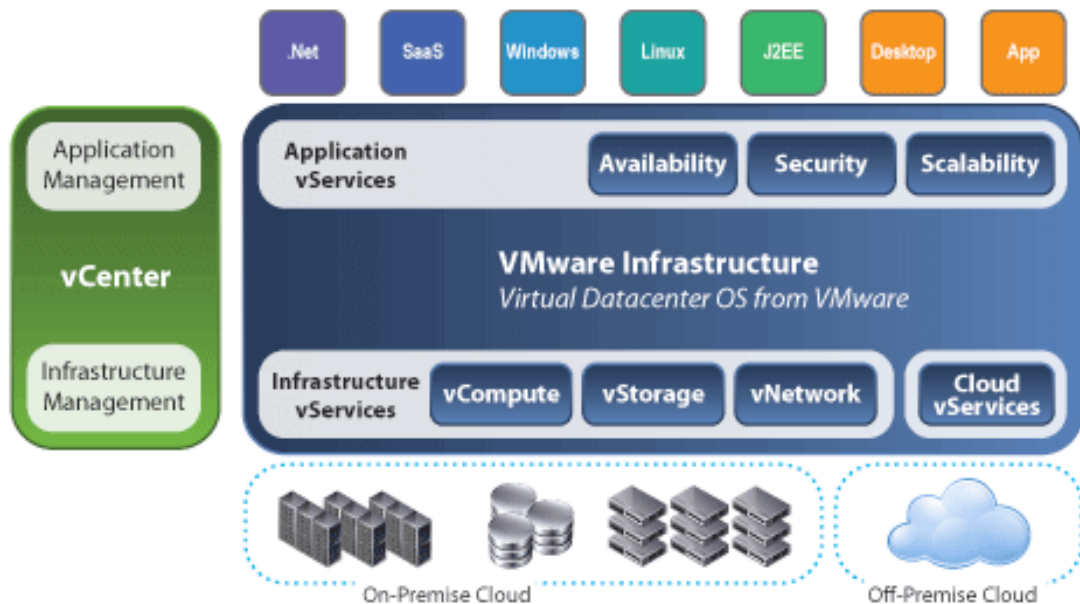
SAN helps separate processing and storage

- Blade chassis for high-density computing tasks
- Typically connected to a SAN, but this is not required
- Exists in half the size visualized to the left

A blade server



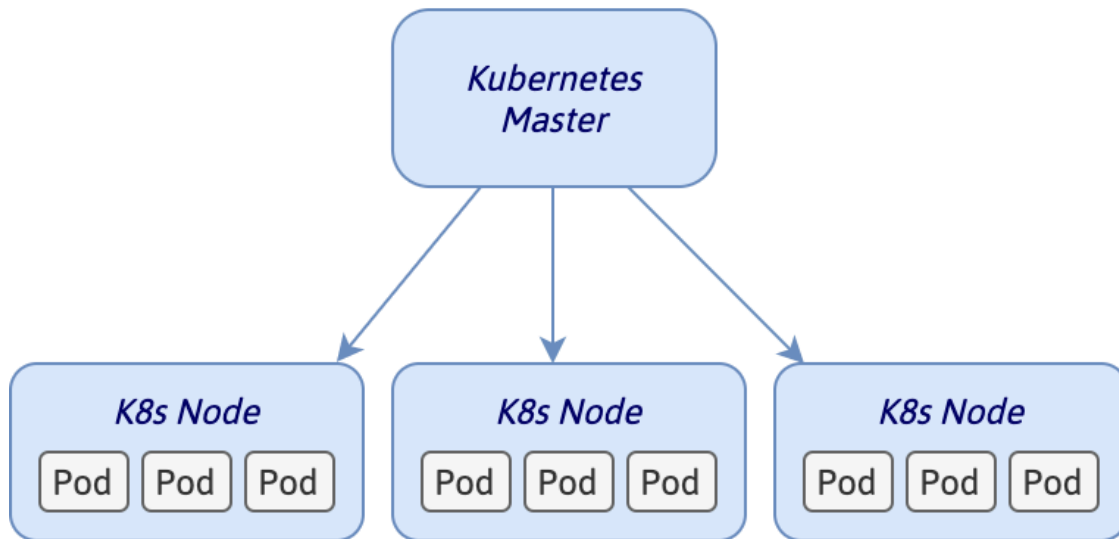
Virtualization — the VMware stack



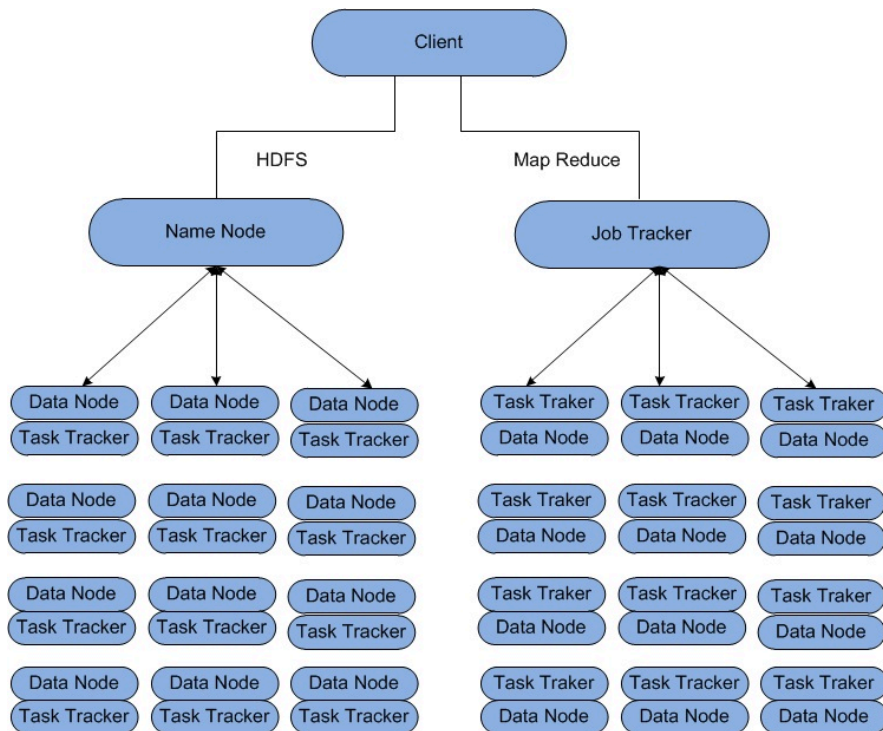
A Kubernetes cluster

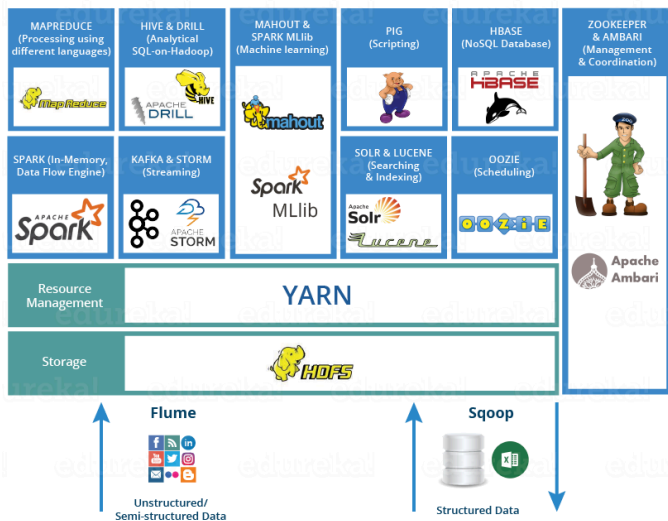


Kubernetes Cluster



Hadoop and MapReduce





DFS — Hadoop

- Google published the **GFS** and **MapReduce** papers in 2003–2004
- Yahoo! was building “Nutch”, an open source web search engine, at the same time
- Yahoo! adopted the technologies pioneered by Google
- Hadoop was primarily driven by Doug Cutting and Tom White in 2006
- It has been evolving ever since...

Project, Groups and Links

Project, Discord Community and GitHub

Project description <https://docs.hviid.phd/bigdataproject>

Discord community <https://discord.gg/B6JzvgJkk9>

Remember to change your name to your real name, or you will not get access.

GitHub <https://github.com/JakobHviidBDDST/BigDataCourseExercises>

Exercise hours and group creation!!!

Next

- Kubernetes access
- Group creation
- And more...

Short break, and now to the fun stuff!